

Reviewer Pack — Symmetric Group Orbit Count Invariant for AC^0 PARITY Circuit...

Entry 28c3cf3ded57 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 09:36:18 UTC

Symmetric Group Orbit Count Invariant for AC^0 PARITY Circuits

Entry ID: 28c3cf3ded57 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 09:36:18 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): AC^0 circuits computing PARITY

Statement:

For any AC^0 circuit C computing PARITY on n variables, the number of distinct irreducible representations of S_n appearing in the decomposition of C 's action is $\Omega(n^{\{1/2\}})$

Rationale (proposer's reasoning):

Symmetric group representations capture combinatorial symmetries of PARITY. The orbit count reflects structural constraints on AC^0 circuits, which cannot exploit high-dimensional symmetry due to depth limitations.

Taxonomy category: AC^0_PARITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0a2bbc56cda6a348

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def binomial_coefficient(n, k):
    return factorial(n) // (factorial(k) * factorial(n - k))

def young_tableau_decomposition(n):
    def hook_length_formula(tableau):
        hook_lengths = []
        for row in range(n):
            for col in range(n):
                hook_lengths.append((n - row) + (n - col) - 1)
        det = 1
        for i in range(n):
            for j in range(i, n):
                det *= hook_lengths[i * n + j]
        return det

    def generate_tableaux():
        tableaux = []
        for i in range(1 <= (n * n)):
            tableau = [[0] * n for _ in range(n)]
            count = 0
            for row in range(n):
                for col in range(n):
                    if i & (1 <= (row * n + col)):
                        tableau[row][col] = count + 1
                        count += 1
            tableaux.append(tableau)
        return tableaux

    tableaux = generate_tableaux()
    orbit_count = sum(1 for t in tableaux if hook_length_formula(t) % n == 0)
    return orbit_count

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n = 40
c = 0.5
instances_tested = 30
orbit_counts = []

for _ in range(instances_tested):
    # Generate a random AC0 circuit computing PARITY on n variables
    circuit = [random.choice([0, 1]) for _ in range(2*n)]

    # Compute the orbit count under Sn action using Young tableau decomposition
    orbit_count = young_tableau_decomposition(n)
    orbit_counts.append(orbit_count)

mean_orbit_count = sum(orbit_counts) / instances_tested
conjecture_holds = all(count >= c * math.sqrt(n) for count in orbit_counts)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "orbit_count",
    "metric_value": mean_orbit_count,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support/falsification criteria | next: Increase time limit and re-run test with optimized code for larger n values

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	96380
2	preregistration	ollama_remote	qwen3:8b	0	0	24243
3	novelty	ollama_remote	qwen3:8b	0	0	20784
4	novelty	ollama_remote	qwen3:8b	0	0	16677
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14546
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9092
7	judge	ollama_remote	qwen3:8b	0	0	63917

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 245640 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/28c3cf3ded57.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/28c3cf3ded57.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/28c3cf3ded57.tar.gz` (if generated)